CUMA Connect v2

Feuille de route détaillée, version 2 : 10 semaines pour un projet vitrine Spring Boot / Kotlin, React, PostgreSQL, Docker, CI/CD, AWS et data

1. Objectif et règles du jeu

- **Résultat final :** un dépôt GitHub public avec README soigné, CI verte, démo en ligne et un récit d'entretien de 2 minutes.
- **Hypothèse de rythme :** 8 à 10 h par semaine, démarrage le lundi 12 octobre 2026, fin vers le 20 décembre. Si tu avances moins vite, décale tout : l'ordre compte plus que les dates.
- **Règle 1 :** tests et code sont écrits ensemble. Pas de fonctionnalité terminée sans au moins un test.
- **Règle 2 :** une branche par tâche (`feat/reservations`), une pull request vers `main` même en solo, merge uniquement si la CI est verte.
- **Règle 3 :** commits courts au format `feat:, fix:, test:, ci:, docs:`. Un historique lisible est un argument en entretien.
- **Règle 4 :** à la fin de chaque phase, un tag Git (`v0.1, v0.2...`) et une ligne dans le README.

Nouveautés de la version 2

Les lignes surlignées en jaune dans les tableaux correspondent à ces changements.

- **Qui utilise quoi :** tout adhérent voit, sur le planning et l'historique, le nom de la personne qui a réservé chaque matériel. Annuler et clôturer restent réservés au propriétaire de la réservation ou à l'admin.
- **Notes sur les jours :** une note rattachée à une date, générale ou liée à un matériel, visible par tous dans le planning. Chacun modifie et supprime les siennes ; l'admin peut tout supprimer.
- **Matériel bloqué par adhérent :** par défaut tout le monde a accès à tout. L'admin bloque un matériel pour un adhérent ; ce matériel **disparaît complètement** pour lui : liste, fiche, planning, notes liées. L'API répond 404, comme si le matériel n'existait pas.
- **Compteur et plusieurs réservations par jour :** chaque réservation a un compteur de début (pré-rempli avec le dernier compteur de la machine) et un compteur de fin. Tant que le compteur de fin n'est pas saisi, la machine est occupée. Dès qu'il l'est, la réservation est clôturée et la machine redevient disponible le reste de la journée. Toutes les réservations d'un même jour sont conservées et affichées l'une sous l'autre.
- **Prix masqués pour les adhérents :** les adhérents ne voient ni tarif, ni montant, ni facture. Le tarif est facultatif et seul l'admin le gère, quand il le souhaite ; la facturation devient un écran admin.

2. Vue d'ensemble

Phase	Semaines	Livrable	Compétences validées
0. Cadrage	Avant S1 (2 h)	Dépôt, board d'issues, outils installés	Git, organisation
1. Socle backend	S1 à S5	API complète, sécurisée, testée, conteneurisée, CI verte (tag v0.1)	Spring Boot, Kotlin, PostgreSQL, JUnit, Docker, GitHub Actions
2. Front	S6 à S7	Interface React complète branchée à l'API (tag v0.2)	TypeScript, React
3. Cloud et data	S8 à S9	S3, Lambda, MongoDB, export pandas, dashboard (tag v0.3)	AWS, NoSQL, pandas, Power BI
4. Finition	S10	Déploiement, README, pitch (tag v1.0)	CD, documentation, communication
Bonus	S11 et plus	Application Android (Compose)	Mineure mobile

3. Architecture cible

Composant	Technologie	Rôle
API	Spring Boot 3, Kotlin, Gradle Kotlin DSL, JDK 21	Règles métier, sécurité, accès aux données
Base principale	PostgreSQL 16, Flyway	Adhérents, matériel, réservations, notes, blocages, factures
Base d'historique	MongoDB	Journal d'événements d'usage (documents)

Composant	Technologie	Rôle
Front	React, TypeScript, Vite, TanStack Query, FullCalendar, Recharts	Planning, réservation, notes, écrans admin, tableau de bord
Stockage	AWS S3 (LocalStack en local)	Photos du matériel, PDF de factures
Notifications	AWS Lambda + EventBridge (+ SES en sandbox)	Rappels de réservation la veille
Qualité	JUnit 5, MockK, Testcontainers, Vitest	Tests unitaires, d'intégration, front
Industrialisation	Docker, Docker Compose, GitHub Actions, GHCR	Build, tests, images, déploiement
Analyse	pandas, Power BI (ou Metabase)	Statistiques d'usage et dashboard

Modèle de données (PostgreSQL)

Table	Colonnes principales
`adherent`	id, nom, email (unique), mot_de_passe_hash, role (ADMIN ou ADHERENT), cree_le
`materiel`	id, nom, categorie, description, unite_compteur (HEURE ou HECTARE), **compteur_actuel**, **tarif_unitaire (nullable, admin seulement)**, photo_key, actif
`reservation`	id, materiel_id, adherent_id, debut, fin (timestamptz), **fin_reelle** (nullable), statut (PLANIFIEE, TERMINEE, ANNULEE), **compteur_debut**, **compteur_fin** (nullable), commentaire
`note_jour`	id, jour (date), materiel_id (nullable = note générale), auteur_id, contenu, cree_le, modifie_le
`blocage_materiel`	adherent_id, materiel_id (clé primaire composée), bloque_par, bloque_le, motif. Une ligne présente = matériel bloqué ; pas de ligne = accès autorisé.
`facture`	id, reservation_id (unique), adherent_id, quantite, montant_ht, emise_le, pdf_key. Créée seulement si le matériel a un tarif ; visible par l'admin uniquement.

Règle clé anti-conflit : contrainte d'exclusion PostgreSQL (extension `btree_gist`) sur `(materiel_id WITH =, tstzrange(debut, COALESCE(fin_reelle, fin)) WITH &&)` WHERE statut <> 'ANNULEE', plus `CHECK (fin > debut)` et `CHECK (fin_reelle IS NULL OR fin_reelle > debut)`. La base garantit l'absence de chevauchement même avec des requêtes simultanées ; l'API traduit la violation en erreur HTTP 409.

Libération du créneau : à la clôture, `fin_reelle` prend l'heure réelle de saisie du compteur de fin. La plage occupée se raccourcit d'elle-même, donc une nouvelle réservation devient possible juste après, le même jour, sans aucun code de vérification en plus.

Règles du compteur : `compteur_debut` est pré-rempli avec `compteur_actuel` de la machine ; `compteur_fin` doit être supérieur ou égal à `compteur_debut` (sinon 400) ; la quantité utilisée vaut compteur_fin - compteur_debut ; la clôture met à jour `materiel.compteur_actuel` dans la même transaction.

Matériel bloqué : le filtrage se fait dans la requête SQL (`NOT EXISTS (SELECT 1 FROM blocage_materiel ...)`), pas après coup en mémoire. Un appel direct à un matériel bloqué renvoie 404, ce qui ne révèle pas son existence.

Endpoints à livrer

Méthode et chemin	Description	Accès
`POST /api/auth/register, /login`	Inscription, connexion, retourne un JWT	Public
`GET /api/materiels, /{id}`	Liste et fiche ; matériel bloqué absent de la liste, 404 sur la fiche ; pas de tarif pour un adhérent	Connecté
`POST PUT DELETE /api/materiels`	Création, modification, suppression du matériel (dont le tarif)	Admin
`POST /api/materiels/{id}/photo`	URL pré-signée S3 pour envoyer une photo	Admin
`GET /api/reservations?materielId&from&to`	Planning filtré par période, avec le **nom de l'adhérent** sur chaque réservation ; matériels bloqués exclus	Connecté
`POST /api/reservations`	Crée une réservation (409 si conflit, 404 si matériel bloqué) ; renvoie le compteur de début proposé	Connecté
`POST /api/reservations/{id}/cancel`	Annule (propriétaire ou admin)	Connecté
`POST /api/reservations/{id}/complete`	Saisie du compteur de fin, libère le créneau ; crée la facture si un tarif existe	Propriétaire ou admin

Méthode et chemin	Description	Accès
`GET POST PUT DELETE /api/notes?from&to`	Notes du jour ; modification par l'auteur, suppression par l'auteur ou l'admin	Connecté
`GET PUT DELETE /api/admin/blocages`	Grille adhérents x matériels : bloquer, débloquer	Admin
`GET /api/admin/factures, /{id}/pdf`	Liste et PDF des factures	Admin
`GET /api/stats/usage?from&to`	Heures et hectares par machine et par adhérent	Admin

Structure du dépôt

```
cuma-connect-v2/
  backend/            Spring Boot Kotlin (src/main, src/test, build.gradle.kts)
  frontend/           React + TypeScript (Vite)
  lambda/             fonction de rappel (Python ou Node)
  analytics/          notebook pandas, export CSV, fichier Power BI
  infra/              docker-compose.yml, docker-compose.dev.yml, scripts de déploiement
  docs/               schéma d'architecture, décisions techniques (ADR), captures
  .github/workflows/  ci.yml, cd.yml
  README.md
```

4. Planning semaine par semaine

Semaine 0 - Cadrage (2 h, avant de commencer)

- [ ] Créer le dépôt `cuma-connect-v2` (public, licence MIT) avec l'arborescence ci-dessus et un README minimal.
- [ ] Créer un board GitHub Projects avec une issue par tâche des semaines 1 à 10 (copie les cases à cocher de ce document).
- [ ] Installer : JDK 21, IntelliJ IDEA, Docker Desktop, Node 20 LTS, Git. Vérifier `docker run hello-world`.
- [ ] Écrire `docs/cahier-des-charges.md` : acteurs (admin, adhérent), 7 user stories principales (dont notes, blocage, compteur), règles du compteur.

Livrable : dépôt initialisé, board rempli.

Validation : tu peux cloner le dépôt sur une autre machine et retrouver la structure.

Semaine 1 - Socle API et base de données

- [ ] Générer le projet avec Spring Initializr : Kotlin, Gradle Kotlin DSL, Java 21, dépendances Web, Data JPA, Validation, PostgreSQL, Flyway, Actuator.
- [ ] Écrire `infra/docker-compose.dev.yml` avec PostgreSQL 16 ; configurer `application.yml` par profils (dev, test, prod).
- [ ] Migration Flyway `V1__init.sql` : tables `adherent` et `materiel` (avec `compteur_actuel` et `tarif_unitaire` nullable).
- [ ] Entités JPA, repositories, DTO (data classes Kotlin) ; deux DTO matériel : un pour l'admin avec tarif, un pour l'adhérent sans.
- [ ] CRUD complet `/api/materiels` avec validation (`@Valid`) et gestion d'erreurs centralisée (`@RestControllerAdvice`, format ProblemDetail).
- [ ] Premiers tests : `@DataJpaTest` avec Testcontainers PostgreSQL, et `@WebMvcTest` sur le contrôleur.

Livrable : CRUD matériel fonctionnel, testé, base lancée par Docker Compose.

Validation : `./gradlew test` passe ; tu peux créer et lister du matériel avec curl ou Postman.

Semaine 2 - Réservations, conflits et compteur de début

- [ ] Migration `V2__reservation.sql` : table `reservation` avec `compteur_debut`, `compteur_fin`, `fin_reelle` ; `CREATE EXTENSION btree_gist` ; contrainte d'exclusion sur `COALESCE(fin_reelle, fin)` et les deux CHECK.
- [ ] `ReservationService` : création (compteur de début pré-rempli depuis `compteur_actuel`), annulation, listing par période et par matériel.
- [ ] Traduire `DataIntegrityViolationException` en réponse 409 avec un message clair (matériel déjà réservé sur ce créneau).
- [ ] Règles métier : pas de réservation dans le passé, durée minimale 1 h, matériel inactif non réservable.

- [ ] Le DTO de réservation contient le nom de l'adhérent (jointure, pas de requête par ligne) : tout le monde voit qui utilise quoi.
- [ ] Tests unitaires du service avec MockK ; test d'intégration qui lance deux réservations en parallèle sur le même créneau et vérifie qu'une seule passe.

Livrable : gestion des réservations avec garantie d'absence de conflit.

Validation : le test de concurrence est stable (lance-le 20 fois) ; couverture du service supérieure à 80 %.

Semaine 3 - Authentification JWT, blocages et première CI

- [ ] Spring Security : inscription, connexion, mots de passe BCrypt, JWT signé (durée 1 h), rôles ADMIN et ADHERENT.
- [ ] Règles d'accès par rôle sur chaque endpoint (tableau de la section 3) ; seul le propriétaire ou l'admin annule ou clôture une réservation.
- [ ] Migration `V3__blocage_materiel.sql` et endpoints `/api/admin/blocages` (lister, bloquer, débloquer).
- [ ] Filtrer les matériels bloqués dans les requêtes de liste, de fiche et de planning ; 404 sur la fiche et à la création d'une réservation.
- [ ] Tests de sécurité avec `spring-security-test` : 401, 403, et un adhérent bloqué qui ne voit pas le matériel (absent de la liste, 404 sur la fiche, 404 en réservation).
- [ ] Documentation OpenAPI avec springdoc (Swagger UI sur `/swagger-ui.html`).
- [ ] Écrire `.github/workflows/ci.yml` : checkout, JDK 21, cache Gradle, `./gradlew build` (tests inclus), rapport de tests en artefact. Ajouter le badge CI au README.

Livrable : API sécurisée avec blocages par matériel, pipeline de CI.

Validation : une PR avec un test cassé est bloquée par la CI ; une PR correcte passe au vert.

Semaine 4 - Compteur de fin et plusieurs réservations par jour

- [ ] Endpoint `complete` : saisie du compteur de fin, contrôle `compteur_fin >= compteur_debut` (400 sinon), `fin_reelle` = maintenant, statut TERMINEE, mise à jour de `compteur_actuel`, le tout en une transaction.
- [ ] Refuser une clôture avant le début de la réservation, ou sur une réservation annulée ou déjà terminée (409).
- [ ] Vérifier que le créneau libéré est réservable aussitôt : une deuxième puis une troisième réservation le même jour sur la même machine.
- [ ] Planning : renvoyer toutes les réservations du jour triées par heure, avec compteurs début et fin.
- [ ] Tests d'intégration : deux réservations le même jour après clôture ; nouvelle réservation refusée tant que la première n'est pas clôturée ; compteur incohérent ; clôture par un autre adhérent (403).

Livrable : cycle complet réservation, utilisation, clôture au compteur.

Validation : scénario rejoué avec curl : réserver 8 h-18 h, clôturer à 11 h, réserver 11 h 30-15 h sur la même machine.

Semaine 5 - Notes du jour, facturation admin, Docker (fin du socle)

- [ ] Migration `V4__note_jour.sql` et CRUD `/api/notes` : note générale ou liée à un matériel ; modification par l'auteur, suppression par l'auteur ou l'admin ; notes liées à un matériel bloqué masquées.
- [ ] Migration `V5__facture.sql`. À la clôture, si le matériel a un tarif, créer la facture (quantité = compteur_fin - compteur_debut, montant = quantité x tarif). Sinon, rien.
- [ ] Endpoints `/api/admin/factures` et PDF de facture (OpenPDF), réservés à l'admin ; stockage local pour l'instant, S3 en semaine 8.
- [ ] Endpoint `/api/stats/usage` : heures et hectares par machine, par adhérent et par mois, calculés depuis les compteurs.
- [ ] Tests : `@ParameterizedTest` sur le calcul (heure, hectare, quantité nulle, arrondis, matériel sans tarif) ; un adhérent n'obtient jamais de champ tarif ou montant.
- [ ] Dockerfile multi-stage du backend (build Gradle puis image JRE), utilisateur non root. Compléter `ci.yml` avec la construction de l'image.
- [ ] Écrire `infra/docker-compose.yml` (API + PostgreSQL) : tout démarre avec `docker compose up`. Poser le tag `v0.1` et une release GitHub.

Livrable : backend complet, conteneurisé, intégré en continu.

Validation : sur une machine vierge, `docker compose up` donne une API fonctionnelle documentée sur Swagger.

Semaine 6 - Front : fondations, authentification, matériel

- [ ] Créer le projet Vite React + TypeScript ; ESLint, Prettier, mode `strict` activé.
- [ ] Générer les types TypeScript depuis l'OpenAPI de l'API (`openapi-typescript`) pour éviter les types écrits à la main.
- [ ] Client HTTP + TanStack Query ; React Router ; page de connexion et inscription ; stockage du JWT et routes protégées.
- [ ] Pages matériel : liste en cartes et fiche détail (sans prix pour les adhérents) ; formulaire admin de création et modification avec tarif facultatif (React Hook Form + Zod).
- [ ] Écran admin des blocages : grille adhérents x matériels, une case à cocher par couple, mise à jour immédiate.
- [ ] Composant de gestion d'erreurs et d'états de chargement réutilisable.

Livrable : front connecté à l'API : connexion, matériel, blocages.

Validation : un admin bloque une machine pour un adhérent ; connecté avec ce compte, la machine n'apparaît nulle part.

Semaine 7 - Front : planning, compteur, notes, tableau de bord

- [ ] Planning FullCalendar : vue semaine par machine, nom de l'adhérent sur chaque réservation, plusieurs réservations par jour affichées l'une sous l'autre, message clair en cas de 409.
- [ ] Formulaire de réservation avec compteur de début pré-rempli ; bouton « Clôturer » qui demande le compteur de fin et libère la suite de la journée.
- [ ] Notes du jour : pastille sur les jours concernés, lecture au survol ou au clic, ajout, modification et suppression selon les droits.
- [ ] Page « mes réservations » (annulation, clôture). Côté admin : liste des factures et téléchargement PDF.
- [ ] Tableau de bord admin avec Recharts : heures par machine, hectares par mois, top adhérents (données de `/api/stats/usage`).
- [ ] Tests Vitest et Testing Library sur 3 composants clés (création de réservation, clôture au compteur, note du jour).
- [ ] Dockerfile du front (build Vite servi par Nginx) ; ajout au Compose ; job front dans la CI (lint, tests, build). Poser le tag `v0.2`.

Livrable : application complète utilisable de bout en bout.

Validation : scénario complet sans Postman : inscription, réservation, clôture au compteur, deuxième réservation le même jour, note, statistiques.

Semaine 8 - AWS : S3 et Lambda

- [ ] Créer un compte AWS, **activer immédiatement une alerte de budget à 1 euro**, créer un utilisateur IAM à droits limités (jamais la clé racine).
- [ ] En local : ajouter LocalStack au Compose pour simuler S3. Dans l'API : AWS SDK v2, URL pré-signées pour envoyer les photos de matériel et lire les PDF de factures.
- [ ] Brancher l'upload dans le front (envoi direct vers S3 via URL pré-signée) et stocker `photo_key` / `pdf_key` en base.
- [ ] Lambda de rappel (dossier `lambda/`) : déclenchée chaque jour par EventBridge, interroge un endpoint interne de l'API pour les réservations du lendemain et envoie un e-mail via SES en sandbox (ou écrit dans CloudWatch si SES bloque).
- [ ] Optionnel en parallèle : préparer la certification AWS Cloud Practitioner (le contenu recoupe cette semaine).

Livrable : fichiers stockés sur S3, une Lambda planifiée qui tourne réellement.

Validation : une photo envoyée depuis l'interface apparaît dans le bucket ; les logs de la Lambda montrent l'exécution quotidienne.

Semaine 9 - NoSQL et data : MongoDB, pandas, dashboard

- [ ] Ajouter MongoDB au Compose. Spring Data MongoDB : collection `evenements_usage` (type, machine, adhérent, date, compteurs, charge utile JSON).
- [ ] Publier des événements applicatifs (`ApplicationEventPublisher`) à la création, l'annulation, la clôture, au blocage et à l'ajout de note ; un listener les écrit dans MongoDB. Documenter dans un ADR pourquoi ce besoin se prête à du document.
- [ ] Endpoint d'export CSV des usages ; notebook `analytics/usage.ipynb` avec pandas : taux d'occupation par machine, heures par mois, nombre de réservations par jour, machines sous-utilisées.
- [ ] Dashboard : Power BI Desktop (Windows uniquement) ou, sinon, Metabase en Docker branché sur PostgreSQL. Exporter une capture dans `docs/`.
- [ ] Poser le tag `v0.3`.

Livrable : journal d'événements, analyse pandas, dashboard décisionnel.

Validation : le notebook s'exécute de bout en bout et produit au moins 3 graphiques exploitables.

Semaine 10 - Déploiement, finition, pitch

- [ ] Choisir une cible simple : petit VPS avec Docker Compose, ou Render / Fly.io. Workflow `cd.yml` : sur tag, publier les images sur GHCR puis déployer.
- [ ] HTTPS (Caddy ou Traefik), variables d'environnement hors dépôt (secrets GitHub), sauvegarde de la base documentée.
- [ ] README final : description en 3 lignes, captures, schéma d'architecture (Mermaid ou image), badges (CI, version), section « Lancer le projet en 3 commandes », stack et choix techniques.
- [ ] Comptes de démo dans le README : un admin, un adhérent, et un adhérent bloqué sur une machine, pour que le recruteur voie les droits en action.
- [ ] Relecture : supprimer le code mort, vérifier qu'aucun secret n'est dans l'historique Git, passer un coup de linter.
- [ ] Rédiger le pitch de 2 minutes et les réponses aux questions probables (voir section 6). Tag `v1.0`.

Livrable : projet en ligne, documenté, prêt à être montré.

Validation : une personne extérieure lance le projet ou ouvre la démo sans ton aide.

Bonus - Semaines 11 et plus : application Android

- [ ] Projet Kotlin + Jetpack Compose, Retrofit, authentification JWT, liste du matériel accessible, création d'une réservation et clôture au compteur depuis le terrain.
- [ ] À ne faire qu'une fois la v1.0 publiée. Si le temps manque, ne le fais pas : il pèse moins que le reste.

5. Critères de réussite et ordre de sacrifice

Le projet est « fini » quand :

- [ ] La CI est verte sur `main` et affiche un badge dans le README.
- [ ] Un `docker compose up` suffit à tout lancer en local.
- [ ] L'API a plus de 35 tests, dont un test de concurrence, un test multi-réservations dans la journée et un test d'adhérent bloqué.
- [ ] Le front couvre le scénario complet sans passer par un outil externe.
- [ ] Aucun écran ni réponse d'API ne montre un prix à un adhérent.
- [ ] Une démo est en ligne avec des comptes de test.
- [ ] Le README contient schéma, captures et instructions vérifiées par quelqu'un d'autre.

Si tu prends du retard, coupe dans cet ordre

1. Application Android (bonus).
2. Power BI / dashboard décisionnel (garde le notebook pandas).
3. MongoDB (documente-le en « évolution prévue »).
4. Lambda (garde S3).
5. PDF de facture (garde le calcul et la liste admin).

Ne coupe jamais : tests, CI/CD, Docker, README, et les quatre fonctionnalités métier (qui utilise quoi, notes, blocages, compteur). Ce sont elles qui font la valeur du projet pour la CUMA.

6. Valoriser le projet

Lignes de CV (à adapter avec tes vrais chiffres)

- Conception et développement d'une API REST Spring Boot / Kotlin (JWT, PostgreSQL, Flyway) pour la gestion de matériel agricole partagé : réservations sans conflit garanties par la base, clôture au compteur, droits d'accès par matériel.
- Mise en place d'une chaîne CI/CD GitHub Actions (tests JUnit et Testcontainers, build Docker, déploiement) et conteneurisation avec Docker Compose.
- Développement d'un front React / TypeScript (planning partagé, notes du jour, administration des accès, tableau de bord) et intégration AWS (S3, Lambda).
- Journalisation d'événements dans MongoDB et analyse d'usage avec pandas et un tableau de bord décisionnel.

Pitch de 2 minutes (trame)

- **Contexte :** pendant mon stage en GAEC, j'ai développé une application Symfony de gestion de matériel partagé (CUMA).
- **Pourquoi la v2 :** la reconstruire avec la stack utilisée en entreprise, et ajouter ce qui manquait aux adhérents : savoir qui a la machine, laisser des notes, gérer les droits par machine, enchaîner plusieurs utilisateurs dans la journée grâce au compteur.
- **Le point technique à raconter :** la contrainte d'exclusion PostgreSQL sur la plage réelle d'utilisation ; saisir le compteur de fin raccourcit la plage et libère le créneau sans code supplémentaire.
- **Le résultat :** projet en ligne, pipeline automatisé, lien GitHub.

Questions d'entretien à préparer

- Pourquoi une contrainte en base plutôt qu'une vérification dans le service ?
- Comment la clôture au compteur libère-t-elle le créneau, et que se passe-t-il si deux personnes clôturent en même temps ?
- Pourquoi répondre 404 plutôt que 403 à un adhérent bloqué, et où se fait le filtrage ?
- Comment garantis-tu qu'un adhérent ne reçoit jamais le tarif dans une réponse JSON ?
- Comment fonctionne ton JWT, et que se passe-t-il à l'expiration ?
- Pourquoi MongoDB pour le journal et pas une table PostgreSQL ?
- Que fait ta CI à chaque push, et que ferais-tu pour la rendre plus rapide ?

Astuce : coche les cases au fur et à mesure et recopie chaque semaine dans une issue GitHub. Un board qui avance régulièrement se voit aussi dans l'historique des commits.